

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication

20250284871

Kind Code

A1

Publication Date

September 11, 2025

Inventor(s)

EAVY; Ryan et al.

SYSTEMS AND METHODS FOR IDENTIFYING AND REMEDiating ARCHITECTURE DESIGN DEFECTS

Abstract

Systems and methods for identifying and remediating architecture design defects are disclosed. In one aspect, a method includes generating a new architecture graph pattern based on an architecture design document of an evaluated architecture; determining a model graph pattern, wherein a shape of the model graph pattern is similar to a shape of the architecture graph pattern; determining, based on a comparison of the shape of the model graph pattern with the shape of the new architecture graph pattern, that the new architecture graph pattern includes a design defect; generating, based on the shape of the model graph pattern, a remediated graph pattern; and determining, based on the differences between the remediated graph pattern and the new architecture graph pattern, a suggested remedial change to the architecture design document.

Inventors: EAVY; Ryan (Chicago, IL), OKPOKO; Olisa Obinna (Brighton, MI)

Applicant: JPMORGAN CHASE BANK, N.A. (New York, NY)

Family ID: 1000008620976

Appl. No.: 19/217497

Filed: May 23, 2025

Related U.S. Application Data

parent US continuation 17653805 20220307 parent-grant-document US 12314646 child US 19217497

Publication Classification

Int. Cl.: G06F30/333 (20200101); G06F18/214 (20230101); G06F30/337 (20200101);
G06F30/398 (20200101); G06F40/20 (20200101); G06N20/00 (20190101)

U.S. Cl.:

CPC **G06F30/333** (20200101); **G06F18/214** (20230101); **G06F30/337** (20200101);
G06F30/398 (20200101); **G06F40/20** (20200101); **G06N20/00** (20190101);

Background/Summary

RELATED APPLICATIONS [0001] This application is related to the following U.S. Patent Applications: [0002] Patent application Ser. No. 17/653,777, filed Mar. 7, 2022, entitled SYSTEMS AND METHODS FOR BUILDING AN ARCHITECTURE KNOWLEDGE GRAPH, and having attorney docket number 052227.500731; [0003] Patent application Ser. No. 17/653,781, filed Mar. 7, 2022, entitled SYSTEMS AND METHODS FOR IDENTIFYING AND REMEDIATING ARCHITECTURE RISK, and having attorney docket number 052227.500732; [0004] Patent application Ser. No. 17/653,788, filed Mar. 7, 2022, entitled SYSTEMS AND METHODS FOR BUILDING A UNIFIED ASSET GRAPH, and having attorney docket number 052227.500733; and [0005] Patent application Ser. No. 17/653,797, filed Mar. 7, 2022, entitled SYSTEMS AND METHODS FOR DETERMINING ARCHITECTURE DRIFT, and having attorney docket number 052227.500734. [0006] The disclosure of each of the applications noted, above, is hereby incorporated, by reference, in its entirety.

BACKGROUND

1. Field of the Invention

[0007] Aspects are generally related to predicting and remediating technology architecture design defects using machine learning applied to a knowledge graph.

2. Description of the Related Art

[0008] Organizations seek to govern the design of technology architectures such that they align with the technology design principles of the organization and reduce the introduction of technical debt, all while still being able to respond to the demands of the business. Quality architectures (such as software architectures) must meet both functional and non-functional requirements. Presently, however, organizations put the highest level of focus and effort on assuring quality implementation of an architecture's functional requirements. To that end, tools exist that allow engineers to address issues related to functional requirements or quality of written code, but there are none that focus on evaluating the design or structure of the technology. Reviewing the design and design changes of a technology architecture prior to implementation is important in the delivery of high-quality products to the business. This is typically a manual process, however, which does not scale, and is often overlooked entirely.

SUMMARY

[0009] In some aspects, the techniques described herein relate to a method of evaluating architecture design, including: generating a new architecture graph pattern based on an architecture design document of an evaluated architecture; determining a model graph pattern, wherein a shape of the model graph pattern is similar to a shape of the architecture graph pattern; determining, based on a comparison of the shape of the model graph pattern with the shape of the new architecture graph pattern, that the new architecture graph pattern includes a design defect; generating, based on the shape of the model graph pattern, a remediated graph pattern; and determining, based on the differences between the remediated graph pattern and the new architecture graph pattern, a suggested remedial change to the architecture design document.

[0010] In some aspects, the techniques described herein relate to a method, wherein the suggested remedial change is generated as a natural language statement.

[0011] In some aspects, the techniques described herein relate to a method, wherein the natural

language statement is formatted as a behavior driven architecture language statement.

[0012] In some aspects, the techniques described herein relate to a method, wherein the suggested remedial change is presented via an electronic interface.

[0013] In some aspects, the techniques described herein relate to a method, wherein the electronic interface is an integrated development environment.

[0014] In some aspects, the techniques described herein relate to a method, wherein the new architecture graph pattern is generated by processing the architecture design document with a natural language processing engine.

[0015] In some aspects, the techniques described herein relate to a method, including: training a machine learning model to recognize the model graph pattern within a knowledge graph, wherein the knowledge graph represents a technology infrastructure of an evaluating organization.

[0016] In some aspects, the techniques described herein relate to a system for evaluating architecture design including at least one server including a processor and a memory, wherein the at least one server is configured for operative communication on a technology infrastructure of an evaluating organization, and wherein instructions stored on the memory instruct the processor to: generate a new architecture graph pattern based on an architecture design document of an evaluated architecture; determine a model graph pattern, wherein a shape of the model graph pattern is similar to a shape of the architecture graph pattern; determine, based on a comparison of the shape of the model graph pattern with the shape of the new architecture graph pattern, that the new architecture graph pattern includes a design defect; generate, based on the shape of the model graph pattern, a remediated graph pattern; and determine, based on the differences between the remediated graph pattern and the new architecture graph pattern, a suggested remedial change to the architecture design document.

[0017] In some aspects, the techniques described herein relate to a system, wherein the suggested remedial change is generated as a natural language statement.

[0018] In some aspects, the techniques described herein relate to a system, wherein the natural language statement is formatted as a behavior driven architecture language statement.

[0019] In some aspects, the techniques described herein relate to a system, wherein the suggested remedial change is presented via an electronic interface.

[0020] In some aspects, the techniques described herein relate to a system, wherein the electronic interface is an integrated development environment.

[0021] In some aspects, the techniques described herein relate to a system, wherein the new architecture graph pattern is generated by processing the architecture design document with a natural language processing engine.

[0022] In some aspects, the techniques described herein relate to a system, wherein instructions stored on the memory instruct the processor to train a machine learning model to recognize the model graph pattern within a knowledge graph, wherein the knowledge graph represents a technology infrastructure of an evaluating organization.

[0023] In some aspects, the techniques described herein relate to a non-transitory computer readable storage medium, including instructions stored thereon for evaluating architecture design, which when read and executed by one or more computers cause the one or more computers to perform steps including: generating a new architecture graph pattern based on an architecture design document of an evaluated architecture; determining a model graph pattern, wherein a shape of the model graph pattern is similar to a shape of the architecture graph pattern; determining, based on a comparison of the shape of the model graph pattern with the shape of the new architecture graph pattern, that the new architecture graph pattern includes a design defect; generating, based on the shape of the model graph pattern, a remediated graph pattern; and determining, based on the differences between the remediated graph pattern and the new architecture graph pattern, a suggested remedial change to the architecture design document.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

[0024] FIG. 1 is a block diagram depicting network infrastructure nodes captured as corresponding architecture graph nodes in a knowledge graph, in accordance with aspects.

[0025] FIG. 2 is a block diagram depicting a knowledge graph generated from a network infrastructure, in accordance with aspects.

[0026] FIG. 3 is a detailed view of a knowledge graph, in accordance with aspects.

[0027] FIG. 4 is a block diagram of a system for generating an intended state dimension of a knowledge graph, in accordance with aspects.

[0028] FIG. 5 is a detailed view of a knowledge graph, in accordance with aspects.

[0029] FIG. 6 is a flow chart for generating a knowledge graph from standard architecture design documents, in accordance with aspects.

[0030] FIG. 7 is a block diagram of a model graph pattern, in accordance with aspects.

[0031] FIG. 8 is a block diagram of a new architecture graph pattern, in accordance with embodiments.

[0032] FIG. 9 is a block diagram of a system for identifying and remediating architecture design defects, in accordance with aspects.

DETAILED DESCRIPTION OF EMBODIMENTS

[0033] Aspects are generally related to predicting and remediating technology architecture design defects using machine learning applied to a knowledge graph.

[0034] In accordance with aspects, machine learning can be employed to evaluate architecture designs. Architecture design intent can be captured in an architecture design document, and this document can be processed by a machine learning engine, including a machine learning model. The machine learning model can be trained to recognize operational architecture patterns that meet both functional and non-functional requirements from an architecture knowledge graph. The architecture knowledge graph can be representative of an organization's technology infrastructure in that the graph can be generated based on architectures in operation within the infrastructure. Based on recognition of patterns that meet the intended requirements of an architecture that is being developed, the ML engine can predict whether the design of the architecture being developed will meet the intended requirements declared in the architecture design document.

[0035] An architecture knowledge graph can be generated from one or more states of an evaluated architecture. A first state can be called the intended state. The intended state represents a technological system functioning entirely as it was designed to function. A second state can be called the functional state. The functional state represents the technological system as it actually functions, e.g., in an operational or production environment.

[0036] Any technological system may be evaluated, and the determined information can be used to generate an architecture knowledge graph. That is, any software program, module, package, etc., may be evaluated for information to include in an architecture knowledge graph. Moreover, firmware, embedded code, etc., may be evaluated. In some cases (particularly when evaluating/determining the functional state of a technological system), an evaluation will take place in conjunction with any hardware that the software is designed to, or actually does, execute on, drive, monitor, enhance, etc. As used herein, an “evaluated architecture” refers to a technological system including any necessary software and hardware of the system that is evaluated for information that will be included in an architecture knowledge graph.

[0037] Once identified, an intended state and a functional state of an evaluated architecture can be structured as a knowledge graph. In accordance with aspects, a knowledge graph can be queried by personnel of an evaluating organization to determine what architectures (i.e., systems, such as software systems) exist in the organization's infrastructure and what those systems' capabilities and

behaviors are. A knowledge graph is an abstraction that organizes real-world knowledge and data. A knowledge graph can integrate determined information from many different data sources and can be used to visualize and explain the determined information, particularly with respect to other information in the knowledge graph.

[0038] A knowledge graph can show and explain relationships between entities. The entities are represented in the knowledge graph as nodes and the relationship between the entities are shown as edges (visualized as connections between the nodes). Labels are used to label the edges and explain the relationship between the nodes. Additionally, knowledge graphs can be used in conjunction with machine learning (ML) in order to infer, or “predict,” previously unknown or undetermined relationships between, and attributes of, the various nodes in the knowledge graph.

[0039] A knowledge graph generated from a determined intended state and a determined functional state of an evaluated architecture can include two dimensions: an intended state dimension that represents the determined intended state of an evaluated architecture, and a functional state dimension that represents the determined functional state of the evaluated architecture. Moreover, a knowledge graph may be generated that represents the intended state dimensions and the functional state dimensions of multiple evaluated architectures. For instance, a knowledge graph can be generated that represents the intended state dimensions and the functional state dimensions of each operational software/hardware solution in an evaluating organization's technology infrastructure. In such an example, each operational software/hardware solution would constitute an evaluated architecture, as further defined herein.

[0040] In accordance with aspects, a knowledge graph as described herein can take the form of a directed acyclic graph.

[0041] The functional state of an evaluated architecture can be determined from the evaluated architecture's observed configuration and functionality. That is, telemetry data gathered from operational architecture and from a technology infrastructure, can be used to determine a functional state of an evaluated architecture. For instance, a functional state of an observed architecture can be based on the evaluated architecture's interactions within its environment (e.g., a production network environment). An exemplary process (i.e., an exemplary node identifying operation) for observing environmental interactions of an evaluated architecture includes using packet capture tools (PCAPS) to understand network flows on a production network of an evaluating organization. The packet captures can be used to determine a functional state of an evaluated architecture based on the contents of the packets and their destination and origin.

[0042] Using packet captures, a network topology can be determined based on network traffic. A determined network topology based on data packets transmitted over a network infrastructure can define network nodes based on the packets transmitted between the nodes. The identified network nodes can represent logical nodes. For instance, hardware may be shared between several evaluated architectures. Each evaluated architecture may execute on a virtual operating system (OS). While the virtual operating systems may share allocated hardware, the virtualized OS may be identified as an independent (logical) network node for evaluation purposes.

[0043] Network nodes identified through packet captures (or through other node identifying operations/techniques, described in more detail, below) can be cast as architecture graph nodes on a knowledge graph, and each graph node can represent an evaluated architecture. That is, identified network nodes, architecture graph nodes and evaluated architectures can represent a 1-1-1 ratio with each other, where each identified network node represents an architecture node on the knowledge graph, which, in turn, represents an evaluated architecture. An architecture graph node, as used herein, represents a system, program, or other logical node (i.e., an architecture) of an evaluating organization on a knowledge graph.

[0044] Network/system nodes may also be determined through various other node identifying operations/techniques. For example, internet protocol (IP) addresses and IP address maps and recorded routes may be used to identify network nodes. Other exemplary aspects include

examining deployment mechanisms, records, logs, etc., in order to determine network nodes. For instance, deployment pipelines can be examined to determine what architectures have been deployed onto an evaluating organization's technology infrastructure.

[0045] Deployment mechanisms may contain data that verifies other sources of network nodes. For instance, a deployment mechanism may specify an IP address that an architecture was deployed to. The IP address, as determined via the pipeline deployment mechanism may verify a network node discovered via a packet capture operation, or vice versa. Once determined and/or verified through any suitable operation, the identified and/or verified network nodes can be cast onto the knowledge graph as corresponding architecture graph nodes.

[0046] In accordance with additional aspects, and as noted above, any suitable operation or process for identifying network nodes for inclusion as corresponding architecture graph nodes in a knowledge graph can be employed. Other exemplary node identifying operations for identifying network nodes include inspecting log files from network operating systems; inspecting/analyzing virtual machine (VM) configuration files; capturing and inspecting/analyzing log files from network routers and switches; examining architecture-as-code (AaC) documentation and/or scripts (described in more detail, below), etc.

[0047] FIG. 1 is a block diagram depicting network infrastructure nodes captured as corresponding architecture graph nodes in a knowledge graph. Network infrastructure **105** represents the technology/computer network infrastructure of an evaluating organization. Network infrastructure **105** includes node **110**, node **112**, and node **114**. Executing on node **110** is evaluated architecture **120**. Executing on node **112** is evaluated architecture **122** and executing on node **114** is evaluated architecture **124**. Each of nodes **110**, **112** and **114** represent a logical node configured for networked communication on network infrastructure **105**. Network infrastructure **105** is configured with appropriate network communication devices such as network switches/routers (not shown), to facilitate networked communication between nodes **110**, **112**, and **114**.

[0048] Evaluated architectures **120**, **122**, and **124** are software components executing on the hardware that has been (either physically or virtually) allocated to nodes **110**, **112**, and **114**, respectively. Evaluated architectures **120**, **122**, and **124** each may be any software system that an evaluating organization wishes to evaluate and capture in an architecture knowledge graph—e.g., an accounting software system, a funds settlement software system, an asset trading software system, an MRP/ERP software system, a CRM software system, a bank ledger software system, etc., etc.

[0049] Architecture knowledge graph **150** is generated using data collected through node identifying operations performed on network infrastructure **105**. Architecture knowledge graph **150** is depicted as a directed acyclic graph in the figures and in accordance with aspects.

[0050] Node identifying operations include any suitable technique for identifying logical network nodes on network infrastructure **105** (as further described herein). Once a node is identified through suitable node identifying operations, the node is included in architecture knowledge graph **150** as an architecture node.

[0051] With continued reference to FIG. 1, through node identifying operations, node **110** including evaluated architecture **120** was discovered and was included in architecture knowledge graph **150** as corresponding architecture node **160**. Likewise, node **112** and evaluated architecture **122** was identified and corresponding architecture node **162** was included in architecture knowledge graph **150**. Similarly, node **114** and evaluated architecture **124** were identified and corresponding node **164** was included in architecture knowledge graph **150**. Accordingly, architecture knowledge graph **150** is a representation, in graph form, of the logical architectures of network infrastructure **105**.

[0052] A knowledge graph generated from network nodes can also include attributes about the network nodes as additional nodes in the graph. Further, edges (i.e., connections representing relationships) can be identified between the architecture nodes and the attribute nodes. Each edge

connecting an attribute node to an architecture node can have a label that explains the relationship represented by the edge. In this way, a robust graphical representation of a functional state of an evaluated architecture can be represented by a knowledge graph.

[0053] Node identifying operations can also be used as attribute identifying operations, in accordance with aspects. That is, results of the techniques discussed herein for identifying network nodes and evaluated architectures can be further examined to identify functional attributes of the identified nodes. For instance, while a packet captured in a packet capture operation may be examined for an origin address and a destination address in order to identify nodes at the origin and the destination, further examination may be carried out to inspect the type of data the packet carries. Based on the origin and destination of the packet, and the type of data therein, attributes about the sending and receiving nodes can be determined.

[0054] Attributes determined based on packet inspection can include dependencies. For example, if a receiving node/evaluated architecture consistently receives a certain type of data from a source node/evaluated architecture, it can be inferred that a dependency exists on the source node/evaluated architecture. Packet capture/inspection may also reveal a particular type of data that can imply a certain data classification. These examples with respect to packet capture/inspection are not meant to be limiting, and other attributes may be determinable through the use of packet capture/inspection used as an attribute identifying operation.

[0055] Likewise, many other attribute identifying operations may be employed to determine attributes and corresponding attribute nodes for inclusion in the knowledge graph. As discussed above with respect to node identification, deployment mechanisms (e.g., deployment pipelines) can provide many attributes of corresponding nodes and evaluated architectures. For instance, a deployment mechanism may indicate what operating system and/or platform a particular evaluated architecture was deployed on. Deployment mechanisms may also provide a version number of the evaluated architecture; an internal (i.e., internal to the evaluating organization) identification number of the evaluated architecture; a hosting platform of the evaluated architecture (i.e., a data center location, number, etc.); and the like.

[0056] An inspection of repositories and source code therein may also provide many attributes of an evaluated architecture. Some attributes that may be determined through repositories and source code include the coding language that an evaluated architecture was written in; dependencies of the evaluated architecture; whether the evaluated architecture is inward facing, outward facing, or both; etc., etc.

[0057] In accordance with aspects, some node and attribute identifying operations may rely on documents that include natural language. Natural language documents can be processed with natural language processing (NLP) engines/algorithms in order to determine both architecture nodes and attribute nodes. Examples of documents/artifacts that can be processed with NLP algorithms include configuration files, property files, project object model (POM) files, VM configuration files, etc.

[0058] POM files are XML files that contain information about a project and configuration details used by software project management tools (e.g., Maven™) to build a software project. Some exemplary attributes that are recorded in POM files include source code location, build information, required software dependencies, dependency scope, packaging information, etc.

[0059] Natural language-formatted documents can offer a reliable source for node/attribute identification, because they are often organized to include a defined set of information. Further, they are often stored in known locations, and therefore can be easily accessed for processing by NLP engines/algorithms. NLP is discussed in further detail, below, with respect to determining an intended state dimension of a knowledge graph. It is contemplated, however, that NLP and the particular NLP techniques discussed herein are equally applicable to generating both a functional state dimension and an intended state dimension of a knowledge graph.

[0060] Identified attributes of architecture nodes can be added to the knowledge graph and edges

can be drawn between the architecture node and its corresponding attributes. Labels can be added to the edges to define the relationship between the architecture node and the attribute node. Label values can be derived from a corresponding determined attribute. For instance, if an attribute of an evaluated architecture is “data classification,” the relationship may be a “has” relationship, and the value of the attribute may be “highly confidential.” In this example, then, the knowledge graph would represent that the evaluated architecture node has a data classification attribute, and the value of that attribute is “highly confidential”. With regards to a natural language, the relationship label can be representative of the verb, or predicate, of a sentence.

[0061] FIG. 2 is a block diagram depicting a knowledge graph generated from a network infrastructure, in accordance with aspects. Attribute identifying operations, as discussed herein, have identified attributes of node **110** and evaluated architecture **120**; node **112** and evaluated architecture **122**; and node **114** and evaluated architecture **124**. These attributes have been included in architecture knowledge graph **150** as attribute nodes **170-179**. Graph edges (i.e., connections between the architecture nodes and the attribute nodes) have also been generated and included in architecture knowledge graph **150**. Graph edges are represented in FIG. 2 as lines/arrows from an architecture node to an attribute node. The depicted edges represent the relationship between an architecture node and an attribute node, in accordance with aspects.

[0062] FIG. 3 is a detailed view of architecture knowledge graph **150**, in accordance with aspects. FIG. 3 shows values of attribute nodes **170-179**. FIG. 3 further shows the assigned labels of the graph edges.

[0063] FIG. 3 depicts the following: attribute node **170** that has a value of “ID_001.” Attribute node **170** is related to evaluated architecture node **160**, and the edge has been assigned label **180**, which has a value of “has internal ID.” In accordance with aspects, and with respect to a natural language presentation of the data in a knowledge graph, labels can take the form of predicates, while architecture nodes serve a subjects and attributes as objects. Consequently, an exemplary relationship captured in architecture knowledge graph **150** can be read as, “architecture node [**160**] has [**180**] internal ID ID_001 [**170**].”

[0064] FIG. 3 further depicts the following attributes determined through attribute identifying operations: attribute node **171** that has a value of “non-confidential;” attribute node **172** that has a value of “Java;” attribute node **173** that has a value of “external;” attribute node **174** that has a value of “low;” attribute node **175** that has a value of “high;” attribute node **176** that has a value of “PII” (personally identifiable information); attribute node **177** that has a value of “public cloud;” attribute node **178** that has a value of “highly confidential;” and attribute **179** that has a value of “3.0.”

[0065] FIG. 3 further depicts determined relationships (edges) between the attribute nodes and the architecture nodes, and the labels assigned to each edge in the architecture knowledge graph **150**. The labels are as follows: label **180** has a value of “has internal ID;” label **181** has a value of “has data classification;” label **182** has a value of “written in;” label **183** has a value of “has dependency;” label **184** has a value of “has risk rating;” label **186** has a value of “includes;” label **187** has a value of “deployed to;” and label **179** has a value of “has version.”

[0066] While some of the figures herein depict graph edges with labels that include an object, it is contemplated that, from a natural, and English, language perspective, objects represent attributes. For instance, in the exemplary relationship depicted in FIG. 3 between architecture node **160**, edge **181**, and attribute node **178**, “data classification” is considered an attribute of architecture node **160** (even though it is depicted as part of the edge label). The attribute “data classification” has a value of “highly confidential.” The relationship is a “has” relationship. In accordance with aspects, then, and as shown in some of the figures herein, an attribute can be shown as part of an edge label, with the attribute's value depicted in the attribute node.

[0067] As depicted in FIG. 2 and FIG. 3, architecture knowledge graph **150** shows a functional state graph dimension that represents a functional, or observed, state of evaluated architectures **120**,

122, and **124** as determined through node identifying operations and attribute identifying operations. In accordance with aspects, and as noted above, an intended state dimension that represents a determined intended state of the evaluated architectures can also be generated and included in a knowledge graph.

[0068] In accordance with aspects, an intended state of an evaluated architecture can be a theoretical state in that it may be determined based on the architectural design of the evaluated technological system. For example, an intended state may be determined based on architectural diagrams, flow charts, sequence diagrams, stated/anticipated inputs, stated/anticipated outcomes, and outputs (which may be based on the anticipated inputs), stated design intentions and goals, etc. That is, the intended state can be determined by evaluating architecture design documents, documentation, and/or artifacts that may have been generated prior to development and/or provisioning of a given evaluated architecture.

[0069] Other examples of information that may be determined from such documentation include related pseudo code, anticipated dependencies, anticipated coding languages and platforms, anticipated hardware environments, known application programming interfaces (APIs) that the evaluated architecture will interact with, other known systems that the evaluated architecture will interact with, etc. This information, and other information discovered in architecture design documents can be used to determine an intended state of an evaluated architecture.

[0070] Any architectural and/or design documentation or artifacts created and/or maintained by an evaluating organization may be evaluated to determine the intended state of an evaluated architecture. Any design documentation and artifacts used by an evaluating organization to determine an intended state of an evaluated architecture is collectively referred to herein as “architecture design documentation,” or “architecture design documents.”

[0071] In some aspects, an evaluating organization may collect and organize a repository of architecture design documentation in order to facilitate ease of access to the design intentions included therein. An architecture design documentation repository may include several different types of architecture design documentation.

[0072] In some aspects, an evaluating organization may define a standard architecture design document format, which may be included in an architecture design documentation repository. A standard architecture design document format may be defined to include components of, and/or information from, many different types of architecture design documentation. That is, a standard architecture design document format may include standardized data fields or entries that describe design aspects and intentions of an evaluated architecture. The standardized data fields/entries may be included in the standard architecture design document due to their importance or relevance in understanding the intended state of an evaluated architecture.

[0073] Exemplary data/information that may be collected in a standard architecture design document includes any information that may be found in any architecture design documentation. That is, a standard architecture design document may include anticipated data inputs, outputs, and other dependencies, a specified coding language, applied design patterns, a required technology platform, required hardware, memory requirements, processing requirements, anticipated network bandwidth, and so on.

[0074] A standard architecture design document may include other information such as an identifier that identifies the project/architecture; an information classification that classifies the type of information produced and or stored; indications of whether the architecture is anticipated to be internal or external facing (or both); a repository location for the code and other components of the architecture; and other organization-specific information.

[0075] A standard design document may include details about the data that the architecture is anticipated to process, such as an anticipated data classification (e.g., non-confidential, confidential, or highly confidential), an anticipated data risk profile (e.g., low, medium, high), and/or anticipated restrictions on hosting platforms or locations.

[0076] In accordance with aspects, a standard architecture design document may be formatted as a natural-language document. A standard architecture design document formatted as a natural-language document may take advantage of natural language processing algorithms in order to determine the contents therein, and in order to employ various machine learning algorithms on the included contents in order to infer relationships and similarities between evaluated systems described in various standard architecture design documentation.

[0077] A standard architecture design document may also take the form of an architecture-as-code (AaC) declaration. The AaC concept includes capturing an intended architectural state of a software system (e.g., an evaluated architecture) in a standardized format. The format may be a natural language format, and the entries can be optimized for processing by an NLP, in accordance with aspects.

[0078] An exemplary standard architecture design document format can have entries that define attributes of a system, and the relationship between the defined attributes and the subject system. Exemplary entries can be in sentence form, and may include a subject, a predicate (or verb) and an object. Exemplary entries in a standard architecture design document may include: [0079] System: System ABC [0080] Has internal ID 00123456. [0081] Is internal facing. [0082] Has data classification confidential. [0083] Is an internally developed application. [0084] Is written in Java. [0085] Uses Spring boot framework. [0086] Has latency/response SLO (service level objective) of <3170 ms. [0087] Has availability SLO of 98% availability. [0088] Has external dependency on XYZ Platform.

[0089] The exemplary standard architecture design document entries, above, can be readily processed by an NLP engine that has been trained on the natural language format (referred to herein as “behavior driven architecture language”) of the entries, in accordance with aspects. Behavior driven architecture language includes assertions or intents about an architecture's design written in a natural language (e.g., written in English). A training file may be provided to an NLP algorithm that associates predicates/verbs/verb phrases within the file's entries as relationships. Objects within the entries can be associated with attributes and attribute definitions. The subject system (e.g., System ABC in the example) can be associated with an evaluated architecture. [0090] An NLP engine that has been trained on the behavior driven architecture language can then process standard architecture design documents to determine the intended state of the subject architecture based on the attributes and relationships declared therein. The NLP engine may then output the determined attributes and relationships in a predetermined and machine-readable file format. Exemplary file formats may include JSON, XML, CSV, etc.

[0091] In accordance with aspects, a knowledge graph can be generated based on a machine-readable file format. A machine-readable file format can include tags or indicators that organize the output of an NLP engine to maintain the subject-verb-object structure (which corresponds to the nodes and relationships of a directed cyclic knowledge graph). For instance, if an NLP engine produces a JSON formatted file for an evaluated architecture (e.g., System ABC), including any determined attributes, values, and relationships, the JSON file can act as a JSON-formatted representation of a knowledge graph (or at least part of a knowledge graph that may include data from many similar JSON files). The contents of the JSON file can be readily formatted, and displayed as, a knowledge graph of the determined intended state of a subject system as declared in a corresponding standard architecture design document. The normalized format of design assertions extracted from architecture design documents and formatted in a machine-readable file format that is represented in a knowledge graph is referred to as behavior graph language.

[0092] In the same manner, architectures, attributes, attribute values, and relationship signifiers determined through attribute identifying operations and node identifying operations can be normalized and formatted in a machine-readable file format (i.e., can be formatted in behavior graph language). Subsequently, a functional state dimension of a knowledge graph can be generated from the machine-readable files.

[0093] FIG. 6 is a flow chart for generating a knowledge graph from standard architecture design documents, in accordance with aspects. At step **602**, standard architecture design documents are generated using a behavior driven architecture language. At step **604**, the standard architecture design documents are processed using an NLP engine that has been trained to process the standard architecture design documents and recognize graph relevant relationships asserted with the behavior driven architecture language in which the design document is written. At step **606**, a machine-readable file is generated from the output of the NLP engine (e.g., a JSON file). At step **608**, a knowledge graph is formatted based on the machine-readable file.

[0094] In other aspects, standard architecture design documents may take any suitable or desirable format. Non-ML algorithmic approaches (for example, file “scraping”) may be used to extract data from structured architecture design documents where NLP is not applicable, or in addition to NLP processing. In accordance with aspects, data scraped from architecture design documents may be later processed by an NLP algorithm or other ML algorithm. In accordance with aspects, standard architecture design documents may also take the form of design data collected in a normalized relational database, or an OLAP database, including reports therefrom.

[0095] In accordance with aspects, architecture design documentation may be processed as described herein, and an intended state dimension of a knowledge graph may be generated based on the data and information obtained from the processed architecture design documentation.

Architecture nodes may be discovered through the processing of the architecture design documentation, and these nodes may be added to a knowledge graph, or a verification process may be undertaken to verify that the discovered architecture nodes have already been added to an existing knowledge graph. Thereafter, intended attributes of an evaluated architecture that have been determined through the processing of architecture design documentation can be added to the knowledge graph. Edges connecting the determined intended attributes to their corresponding architecture nodes can also be added to the knowledge graph, and corresponding labels may, in turn, be added to the edges.

[0096] FIG. 4 is a block diagram of a system for generating an intended state dimension of a knowledge graph, in accordance with aspects. FIG. 4 depicts architecture documentation repository **410**, and intent processing engine **412**. Intent processing engine **412** may include NLP engine **414**. Architecture documentation repository **410** stores architecture design documents as described herein. Architecture documentation repository **410** may be any suitable data store, such as a relational data base (e.g., a relational database configured to store architecture design documents as binary large objects (BLOBs)), a flat-file architecture, a folder structure, etc.

[0097] Intent processing engine **412** is configured to access architecture documentation repository **410** and process the architecture design documents therein. Intent processing engine **412** may include NLP engine **414**, which, in turn, may include NLP processing algorithms. Intent processing engine **412** may further include other ML algorithms, or non-ML processing algorithms configured to process the architecture design documents, as described in further detail herein.

[0098] With continued reference to FIG. 4, shown is architecture knowledge graph **150**.

Architecture knowledge graph **150** is shown as including the determined functional state dimension (as described with respect to FIGS. 1-3, above), in accordance with aspects. It is contemplated, however, that a determined functional state dimension need not be previously determined or included in architecture knowledge graph **150** in order to determine an intended state and include that state as an intended state dimension in the knowledge graph. Architecture knowledge graph **150** is depicted as having an intended state dimension included. In accordance with aspects, the intended state dimension, including intended attributes of architecture attributes **160**, **162**, and **164** have been determined by intent processing engine **412** based on architecture design documentation accessed from architecture documentation repository **410**.

[0099] The intended state dimension that has been included in architecture knowledge graph **150** is shown in broken lines, to help distinguish it from the functional state dimension (that is shown in

solid lines) and from the architecture node (shown in solid lines, and shaded). Accordingly, as shown in architecture knowledge graph **150** as depicted in FIG. **4**, the determined intended state dimension includes intended attribute node **190**, intended attribute node **191**, intended attribute node **194**, intended attribute node **195**, intended attribute node **197** and intended attribute node **198**. The intended state dimension further includes corresponding edges indicating relationships with corresponding architecture nodes.

[0100] FIG. **5** is a detailed view of a knowledge graph, in accordance with aspects. FIG. **5** depicts a detailed view of architecture knowledge graph **150**, including both an intended state dimension and a functional state dimension. The intended state dimension is shown in broken lines and the functional state dimension is shown in solid lines. The architecture nodes are shown in solid lines and are shaded.

[0101] In FIG. **5**, the determined intended attributes of architecture node **160** include intended attribute node **190** that has a value of “ID_001;” intended attribute node **195** that has a value of “medium;” intended attribute node **197** that has a value of “private cloud;” and intended attribute node **198** that has a value of “confidential.” Likewise, intended attributes nodes of architecture node **164** have been determined and include intended attribute node **191** that has a value of “non-confidential;” and intended attribute node **194** that has a value of “low.” Edges corresponding to the determined intended attribute nodes have also been determined and included in architecture knowledge graph **150**.

[0102] Because of space constraints of FIG. **5**, the edges of the intended state dimension are not shown with labels. However, the edges of the intended state dimension are assigned the same label as their corresponding functional state attributes. That is, the edge connecting intended attribute node **190** has been assigned label **180** (i.e., with a value of “has internal ID”); the edge connecting intended attribute node **191** has been assigned label **181**; the edge connecting intended attribute node **194** has been assigned label **184**; the edge connecting intended attribute node **195** has been assigned label **184**; the edge connecting intended attribute node **197** has been assigned label **187**; and the edge connecting intended attribute node **198** has been assigned label **181**. Due to space constraints of FIG. **5**, no intended attribute nodes have been shown for architecture node **162**, however, it is contemplated that the determining processes as described herein would have determined intended attribute nodes and corresponding edges and would have included them in architecture knowledge graph **150**.

[0103] In accordance with aspects, an architecture knowledge graph, as described above, captures information about an evaluating organization's technology infrastructure and the software architectures executing on that infrastructure. In accordance with aspects, machine learning (ML) algorithms can process an architecture knowledge graph to determine patterns within the graph.

[0104] As described above, an intended state dimension of an architecture knowledge graph can be based, at least in part, on architecture design documents. Some or all of the architecture design documents may be standard architecture design documents. By building an architecture knowledge graph through architecture design documentation, the design patterns of evaluated architectures can be captured in the architecture knowledge graph. For instance, a standard architecture design document for a microservice can reflect design patterns that an evaluating organization has standardized for microservices executing on its technology infrastructure. Moreover, by employing telemetry data to build a functional state dimension of an architecture knowledge graph, a broader shape, or pattern, of a given architecture can be recognized.

[0105] For example, in an architecture knowledge graph representative of a technology infrastructure with a large number of microservice applications, the nodes and edges that represent each microservice application will tend to have a similar shape, or form a similar pattern, based on a microservice design pattern adopted by the evaluating organization. That is, the microservice design pattern adopted by an evaluating organization can be included (i.e., declared) in its architecture design documentation (e.g., using behavior driven architecture language), and that

design pattern can be reflected in a representative architecture knowledge graph (e.g., in an intended state dimension of the architecture knowledge graph). Exemplary characteristics that define an evaluated architecture's graph pattern can include a technology stack, a design pattern, known dependencies, and other attributes determined as described, above.

[0106] Dependencies can broaden the shape or pattern of an evaluated architecture. Telemetry data collected and used in the formation of a functional state dimension of an architecture knowledge graph can be representative of dependencies not recorded in architecture design documentation. For instance, data flows into and out of evaluated architectures (which may be captured through PCAPS, or other observations, as discussed above) can further broaden an evaluated architecture's shape/pattern in the architecture knowledge graph.

[0107] In accordance with aspects, an ML model, such as a neural network model, can process an architecture knowledge graph to identify patterns of an evaluated architecture. These graph patterns of evaluated architectures are referred to herein as, “evaluated architecture graph patterns,” or simply “graph patterns.” Exemplary ML models for processing and learning knowledge graphs include graph neural network variants such as a graph convolutional network (GCN), a graph attention network (GAT), a graph recurrent network (GRN), etc. However, these examples are not meant to be limiting, and any suitable machine learning model may be employed in aspects disclosed herein.

[0108] Because graph patterns of evaluated architectures are determined based on both intended and functional attributes of the evaluated architecture, a graph pattern can include not only the intended design attributes of an evaluated architecture, but it can also include functional attributes that will indicate whether the intended attributes are being realized. That is, given a graph pattern of an evaluated architecture, an amount of architecture drift of a functional state from an intended state can be determined. Given that an amount of architecture drift can be determined from a graph pattern of an evaluated architecture, the represented evaluated architecture can be determined to have a high level of conformance to its intended state, a low level of conformance to its intended state, an intermediate level of conformance, or some other level of conformance.

[0109] In accordance with aspects, graph patterns representing evaluated architectures that have a high level of conformance to their intended state, and whose intended state represents a design pattern that is known to be based on good, industry-standard, and/or robust design principles, and that further have been determined to meet their non-functional requirements can be learned and recognized by a ML model to be model graph patterns. As used herein, the term “model graph pattern” means an identified graph pattern (also referred to as a graph shape”) that represents an evaluated architecture that is substantially free of design defects. Design defects, as used herein, means an architecture design that does not meet its declared functional or non-functional requirements, that does not adhere to known good, industry standard, and/or organizationally adopted design patterns, and/or that does not conform to its intended state.

[0110] As is typical in most ML model training, a level of supervision may be required when first training an ML model to recognize model graph patterns. That is, a ML model may need preliminary outside input to define a model graph pattern. Typically, however, the amount of training/tuning needed decreases as the model processes larger amounts of related data.

[0111] Once an ML model has been trained to recognize model graph patterns, graph patterns recognized as model graph patterns can, in turn, be used to predict whether new design architectures include architecture design defects or are substantially free from architecture design defects through comparison. A new architecture design pattern for an evaluated architecture can be declared in an architecture design document, and can be processed (e.g., using an NLP engine) to determine an intended state dimension graph pattern for the evaluated architecture. Intended state dimension graph patterns generated for new architecture design patterns are referred to herein as new architecture graph patterns.

[0112] An architecture linter working in conjunction with a trained ML model and ML engine can

compare a new architecture graph pattern to model graph patterns that have a similar shape as the shape of the new architecture graph pattern. If the compared shapes are highly similar, this can indicate that the new architecture graph pattern is free from design defects. If, however, the new architecture graph pattern is dissimilar in aspects, this can indicate that the new architecture graph pattern includes design defects. An architecture linter can further formulate remedial design changes, based on the similar model graph pattern, which can be implemented in the new architecture graph pattern in order to eliminate detected design defects.

[0113] FIG. 7 is a block diagram of a model graph pattern, in accordance with aspects. Model graph pattern **750** includes architecture graph node **760** and attribute nodes **770-777**. Model graph pattern **750** can depict determined intended and functional state dimensions of the evaluated architecture represented by architecture graph node **760**. In accordance with aspects, model graph pattern **750** has been recognized from an evaluating organization's architecture knowledge graph (e.g., knowledge graph **150**) by an appropriately trained ML model as a model graph pattern.

[0114] FIG. 8 is a block diagram of a new architecture graph pattern, in accordance with embodiments. New architecture graph pattern **850** represents a graph pattern of a new architecture design pattern that has been designed by, e.g., software architects. For instance, new architecture graph pattern **850** may represent a new microservice design pattern. New architecture graph pattern **850** can be captured in a standard architecture design document that is stored in an architecture documentation repository. New architecture graph pattern **850** can be processed by an intent processing engine, including an NLP engine, in order to produce a representative graph pattern of an intended state dimension of the new architecture design pattern developed by technology architects.

[0115] In accordance with aspects, an architecture linter engine can evaluate a new architecture graph pattern generated by an intent processing engine in order to determine if the new architecture graph pattern is similar to a model graph pattern that is recognized by an ML engine. The ML engine may be trained to recognize model graph patterns and may process an evaluating organization's architecture knowledge graph to identify model graph patterns therein. The architecture linter engine may be in operative communication with the ML engine and may determine a model graph pattern that is recognized by the ML engine and that is similar to the new architecture graph pattern.

[0116] After determining model pattern that is similar to the new architecture graph pattern, the architecture linter engine can determine, based on the similar model graph pattern, whether the new architecture graph pattern will realize its functional and non-functional requirements (i.e., its declared intended state). If it is determined that the new architecture graph pattern will not realize its requirements, the architecture linter can suggest remedial design changes based on the similar model graph pattern.

[0117] FIG. 9 is a block diagram of a system for identifying and remediating architecture design defects, in accordance with aspects. With reference to FIG. 9, new architecture graph pattern **850** is generated by intent processing engine **412**, which includes NLP engine **414**. NLP engine **414** processes architecture design documents stored in architecture documentation repository **410**. Based on a new architecture design document from architecture documentation repository **410**, intent processing engine **412** produces new architecture graph pattern **850**.

[0118] With continued reference to FIG. 9, ML model **1052** can be trained to recognize model graph patterns in knowledge graph **150**. ML engine **1050** can process knowledge graph **150**, and ML model **1052** can identify model graph pattern **750**. Architecture linter engine **1014** can evaluate new architecture graph pattern **850** and can determine that model graph pattern **750** is similar in shape to that of new architecture graph pattern **850**. Based on differences between model graph pattern **750** and new architecture graph pattern **850**, architecture linter engine **1014** can determine whether new architecture graph pattern **850** includes design defects. If, based on difference in shape from model graph pattern **750**, it is determined that new architecture graph pattern **850**

includes design defects, architecture linter engine **1014** can suggest remedial design changes based on model graph pattern **750**.

[0119] Remediated graph pattern **1075** of FIG. **9** includes remedial changes to new architecture graph pattern **850**. The remedial changes are based on model graph pattern **750** and alter the shape of new architecture graph pattern **850** so that it is more similar to that of model graph pattern **750**. The remedial changes can take the form of additional attribute nodes, less attribute nodes, or different attribute nodes. Architecture linter engine **1014** can generate, based on the graph remedial changes, suggested changes to declarations in the architecture design document of the evaluated architecture. That is, architecture linter **1014** can determine natural language declaration suggestions that, when entered into the architecture design document of the evaluated architecture represented by new architecture graph pattern **850**, would transform the shape of new architecture graph pattern **850** to that of remediated graph pattern **1075**.

[0120] In accordance with embodiments, architecture linter **1014** may be in operative communication with NLP engine **414**, which may be configured to generate the natural language declarations (e.g., in behavior driven architecture language) based in the remedial changes formed remediated graph pattern **1075**. Architecture linter may be further configured to alert architects of the suggested remedial changes. For instance, architecture linter may be in operative communication with an integrated development environment (IDE) used by design architects to generate architecture design documents via, e.g., an API and an IDE plug-in. Architecture linter **1014** can send the suggested remedial changes to the IDE of the design architects.

[0121] In accordance with aspects, the architecture linting process described above may take place in real time, or close to real time. That is, as a design architect builds out a new design by adding design declarations to an architecture design document, the process described above with reference to FIGS. **7-9** may be taking place in the background, such that the design architect may be receiving real-time feedback in the IDE being used to build the architecture design document. In other aspects, the process may be carried out on demand, i.e., in response to an input request for evaluation of the architecture design document. The input request may be received, e.g., in the IDE and passed to the architecture linter engine, which may begin the architecture linting process described above.

[0122] The various processing steps and/or data flows depicted in the figures and described in greater detail herein may be accomplished using some or all of the system components also described herein. In some implementations, the described logical steps may be performed in different sequences and various steps may be omitted. Additional steps may be performed along with some or all of the steps shown in the depicted logical flow diagrams. Some steps may be performed simultaneously. Accordingly, the logical flows illustrated in the figures and described in greater detail herein are meant to be exemplary and, as such, should not be viewed as limiting. These logical flows may be implemented in the form of executable instructions stored on a machine-readable storage medium and/or in the form of electronic circuitry.

[0123] Hereinafter, general aspects of implementation of the systems and methods of the invention will be described.

[0124] The system of the invention or portions of the system of the invention may be in the form of a “processing machine,” such as a general-purpose computer, for example. As used herein, the term “processing machine” is to be understood to include at least one processor that uses at least one memory. The at least one memory stores a set of instructions. The instructions may be either permanently or temporarily stored in the memory or memories of the processing machine. The processor executes the instructions that are stored in the memory or memories in order to process data. The set of instructions may include various instructions that perform a particular task or tasks, such as those tasks described above. Such a set of instructions for performing a particular task may be characterized as a program, software program, or simply software.

[0125] In one embodiment, the processing machine may be a specialized processor.

[0126] As noted above, the processing machine executes the instructions that are stored in the memory or memories to process data. This processing of data may be in response to commands by a user or users of the processing machine, in response to previous processing, in response to a request by another processing machine and/or any other input, for example.

[0127] As noted above, the processing machine used to implement the invention may be a general-purpose computer. However, the processing machine described above may also utilize any of a wide variety of other technologies including a special purpose computer, a computer system including, for example, a microcomputer, mini-computer or mainframe, a programmed microprocessor, a micro-controller, a peripheral integrated circuit element, a CSIC (Customer Specific Integrated Circuit) or ASIC (Application Specific Integrated Circuit) or other integrated circuit, a logic circuit, a digital signal processor, a programmable logic device such as a FPGA, PLD, PLA or PAL, or any other device or arrangement of devices that is capable of implementing the steps of the processes of the invention.

[0128] The processing machine used to implement the invention may utilize a suitable operating system. Thus, embodiments of the invention may include a processing machine running the iOS operating system, the OS X operating system, the Android operating system, the Microsoft Windows™ operating systems, the Unix operating system, the Linux operating system, the Xenix operating system, the IBM AIX™ operating system, the Hewlett-Packard UX™ operating system, the Novell Netware™ operating system, the Sun Microsystems Solaris™ operating system, the OS/2™ operating system, the BeOS™ operating system, the Macintosh operating system, the Apache operating system, an OpenStep™ operating system or another operating system or platform.

[0129] It is appreciated that in order to practice the method of the invention as described above, it is not necessary that the processors and/or the memories of the processing machine be physically located in the same geographical place. That is, each of the processors and the memories used by the processing machine may be located in geographically distinct locations and connected so as to communicate in any suitable manner. Additionally, it is appreciated that each of the processor and/or the memory may be composed of different physical pieces of equipment. Accordingly, it is not necessary that the processor be one single piece of equipment in one location and that the memory be another single piece of equipment in another location. That is, it is contemplated that the processor may be two pieces of equipment in two different physical locations. The two distinct pieces of equipment may be connected in any suitable manner. Additionally, the memory may include two or more portions of memory in two or more physical locations.

[0130] To explain further, processing, as described above, is performed by various components and various memories. However, it is appreciated that the processing performed by two distinct components as described above may, in accordance with a further embodiment of the invention, be performed by a single component. Further, the processing performed by one distinct component as described above may be performed by two distinct components. In a similar manner, the memory storage performed by two distinct memory portions as described above may, in accordance with a further embodiment of the invention, be performed by a single memory portion. Further, the memory storage performed by one distinct memory portion as described above may be performed by two memory portions.

[0131] Further, various technologies may be used to provide communication between the various processors and/or memories, as well as to allow the processors and/or the memories of the invention to communicate with any other entity, i.e., so as to obtain further instructions or to access and use remote memory stores, for example. Such technologies used to provide such communication might include a network, the Internet, Intranet, Extranet, LAN, an Ethernet, wireless communication via cell tower or satellite, or any client server system that provides communication, for example. Such communications technologies may use any suitable protocol such as TCP/IP, UDP, or OSI, for example.

[0132] As described above, a set of instructions may be used in the processing of the invention. The set of instructions may be in the form of a program or software. The software may be in the form of system software or application software, for example. The software might also be in the form of a collection of separate programs, a program module within a larger program, or a portion of a program module, for example. The software used might also include modular programming in the form of object-oriented programming. The software tells the processing machine what to do with the data being processed.

[0133] Further, it is appreciated that the instructions or set of instructions used in the implementation and operation of the invention may be in a suitable form such that the processing machine may read the instructions. For example, the instructions that form a program may be in the form of a suitable programming language, which is converted to machine language or object code to allow the processor or processors to read the instructions. That is, written lines of programming code or source code, in a particular programming language, are converted to machine language using a compiler, assembler or interpreter. The machine language is binary coded machine instructions that are specific to a particular type of processing machine, i.e., to a particular type of computer, for example. The computer understands the machine language.

[0134] Any suitable programming language may be used in accordance with the various embodiments of the invention. Illustratively, the programming language used may include assembly language, Ada, APL, Basic, C, C++, COBOL, dBase, Forth, Fortran, Java, Modula-2, Pascal, Prolog, REXX, Visual Basic, and/or JavaScript, for example. Further, it is not necessary that a single type of instruction or single programming language be utilized in conjunction with the operation of the system and method of the invention. Rather, any number of different programming languages may be utilized as is necessary and/or desirable.

[0135] Also, the instructions and/or data used in the practice of the invention may utilize any compression or encryption technique or algorithm, as may be desired. An encryption module might be used to encrypt data. Further, files or other data may be decrypted using a suitable decryption module, for example.

[0136] As described above, the invention may illustratively be embodied in the form of a processing machine, including a computer or computer system, for example, that includes at least one memory. It is to be appreciated that the set of instructions, i.e., the software for example, that enables the computer operating system to perform the operations described above may be contained on any of a wide variety of media or medium, as desired. Further, the data that is processed by the set of instructions might also be contained on any of a wide variety of media or medium. That is, the particular medium, i.e., the memory in the processing machine, utilized to hold the set of instructions and/or the data used in the invention may take on any of a variety of physical forms or transmissions, for example. Illustratively, the medium may be in the form of paper, paper transparencies, a compact disk, a DVD, an integrated circuit, a hard disk, a floppy disk, an optical disk, a magnetic tape, a RAM, a ROM, a PROM, an EPROM, a wire, a cable, a fiber, a communications channel, a satellite transmission, a memory card, a SIM card, or other remote transmission, as well as any other medium or source of data that may be read by the processors of the invention.

[0137] Further, the memory or memories used in the processing machine that implements the invention may be in any of a wide variety of forms to allow the memory to hold instructions, data, or other information, as is desired. Thus, the memory might be in the form of a database to hold data. The database might use any desired arrangement of files such as a flat file arrangement or a relational database arrangement, for example.

[0138] In the system and method of the invention, a variety of “user interfaces” may be utilized to allow a user to interface with the processing machine or machines that are used to implement the invention. As used herein, a user interface includes any hardware, software, or combination of hardware and software used by the processing machine that allows a user to interact with the

processing machine. A user interface may be in the form of a dialogue screen for example. A user interface may also include any of a mouse, touch screen, keyboard, keypad, voice reader, voice recognizer, dialogue screen, menu box, list, checkbox, toggle switch, a pushbutton or any other device that allows a user to receive information regarding the operation of the processing machine as it processes a set of instructions and/or provides the processing machine with information. Accordingly, the user interface is any device that provides communication between a user and a processing machine. The information provided by the user to the processing machine through the user interface may be in the form of a command, a selection of data, or some other input, for example.

[0139] As discussed above, a user interface is utilized by the processing machine that performs a set of instructions such that the processing machine processes data for a user. The user interface is typically used by the processing machine for interacting with a user either to convey information or receive information from the user. However, it should be appreciated that in accordance with some embodiments of the system and method of the invention, it is not necessary that a human user actually interact with a user interface used by the processing machine of the invention. Rather, it is also contemplated that the user interface of the invention might interact, i.e., convey and receive information, with another processing machine, rather than a human user. Accordingly, the other processing machine might be characterized as a user. Further, it is contemplated that a user interface utilized in the system and method of the invention may interact partially with another processing machine or processing machines, while also interacting partially with a human user.

[0140] It will be readily understood by those persons skilled in the art that the present invention is susceptible to broad utility and application. Many embodiments and adaptations of the present invention other than those herein described, as well as many variations, modifications, and equivalent arrangements, will be apparent from or reasonably suggested by the present invention and foregoing description thereof, without departing from the substance or scope of the invention.

[0141] Accordingly, while the present invention has been described here in detail in relation to its exemplary embodiments, it is to be understood that this disclosure is only illustrative and exemplary of the present invention and is made to provide an enabling disclosure of the invention. Accordingly, the foregoing disclosure is not intended to be construed or to limit the present invention or otherwise to exclude any other such embodiments, adaptations, variations, modifications, or equivalent arrangements.

Claims

1-15. (canceled)

16. A method of evaluating architecture design, comprising: generating a new architecture graph pattern based on an architecture design document of an evaluated architecture and based on one or more environmental interactions found using node identifying operations including identifying a type of data, the new architecture graph pattern comprising a first node, a second node, and an edge connecting the first node and the second node, the edge describing a relationship between the first node and the second node, wherein a first machine learning model infers an unknown relationship between the first node and the second node; determining a model graph pattern, wherein a shape of the model graph pattern is similar to a shape of the architecture graph pattern; determining, based on a comparison of the shape of the model graph pattern with the shape of the new architecture graph pattern, that the new architecture graph pattern includes a design defect; generating, based on the shape of the model graph pattern, a remediated graph pattern; and determining, based on the differences between the remediated graph pattern and the new architecture graph pattern, a suggested remedial change to the architecture design document.

17. The method of claim 16, wherein the suggested remedial change is generated as a natural language statement.

18. The method of claim 17, wherein the natural language statement is formatted as a behavior driven architecture language statement.
19. The method of claim 16, wherein the suggested remedial change is presented via an electronic interface.
20. The method of claim 19, wherein the electronic interface is an integrated development environment.
21. The method of claim 16, wherein the new architecture graph pattern is generated by processing the architecture design document with a natural language processing engine.
22. The method of claim 1, comprising: training a second machine learning model to recognize the model graph pattern within a knowledge graph, wherein the knowledge graph represents a technology infrastructure of an evaluating organization.
23. A method of evaluating architecture design, comprising: generating a new architecture graph pattern based on an architecture design document of an evaluated architecture and based on one or more environmental interactions found using node identifying operations including identifying a origin and destination address, the new architecture graph pattern comprising a first node, a second node, and an edge connecting the first node and the second node, the edge describing a relationship between the first node and the second node, wherein a first machine learning model infers an unknown relationship between the first node and the second node; determining a model graph pattern, wherein a shape of the model graph pattern is similar to a shape of the architecture graph pattern; determining, based on a comparison of the shape of the model graph pattern with the shape of the new architecture graph pattern, that the new architecture graph pattern includes a design defect; generating, based on the shape of the model graph pattern, a remediated graph pattern; and determining, based on the differences between the remediated graph pattern and the new architecture graph pattern, a suggested remedial change to the architecture design document.
24. The method of claim 23, wherein the suggested remedial change is generated as a natural language statement.
25. The method of claim 24, wherein the natural language statement is formatted as a behavior driven architecture language statement.
26. The method of claim 23, wherein the suggested remedial change is presented via an electronic interface.
27. The method of claim 26, wherein the electronic interface is an integrated development environment.
28. The method of claim 23, wherein the new architecture graph pattern is generated by processing the architecture design document with a natural language processing engine.
29. The method of claim 23, wherein instructions stored on the memory instruct the processor to train a second machine learning model to recognize the model graph pattern within a knowledge graph, wherein the knowledge graph represents a technology infrastructure of an evaluating organization.
30. A method of evaluating architecture design, comprising: generating a new architecture graph pattern based on an architecture design document of an evaluated architecture and based on one or more environmental interactions found using deployment pipeline information comprising a deployment pipeline's version number, a deployment pipeline's internal identification number, or a deployment pipeline's hosting platform; the new architecture graph pattern comprising a first node, a second node, and an edge connecting the first node and the second node, the edge describing a relationship between the first node and the second node; wherein a first machine learning model infers an unknown relationship between the first node and the second node; determining a model graph pattern, wherein a shape of the model graph pattern is similar to a shape of the architecture graph pattern; determining, based on a comparison of the shape of the model graph pattern with the shape of the new architecture graph pattern, that the new architecture graph pattern includes a design defect; generating, based on the shape of the model graph pattern, a remediated graph

pattern; and determining, based on the differences between the remediated graph pattern and the new architecture graph pattern, a suggested remedial change to the architecture design document.

31. The method of claim 23, wherein the suggested remedial change is generated as a natural language statement.

32. The method of claim 24, wherein the natural language statement is formatted as a behavior driven architecture language statement.

33. The method of claim 23, wherein the suggested remedial change is presented via an electronic interface.

34. The method of claim 26, wherein the electronic interface is an integrated development environment.

35. The method of claim 23, wherein the new architecture graph pattern is generated by processing the architecture design document with a natural language processing engine.

36. The method of claim 23, wherein instructions stored on the memory instruct the processor to train a second machine learning model to recognize the model graph pattern within a knowledge graph, wherein the knowledge graph represents a technology infrastructure of an evaluating organization.
